PHITRON

Module 8 — Assignment 01

Movie Collection API

Topics: GET • POST • PUT • DELETE • Pydantic Data Validation • SQLite Database • Path Parameters • Query Parameters

Total Marks: 100

Overview

You will build a Movie Collection API from scratch using FastAPI. This time, you will use a SQLite database for persistent storage instead of an in-memory list, and Pydantic models to validate all incoming request data.

Every endpoint should read from and write to the SQLite database, not a Python list or a JSON file.

Data Model

Each movie has the following fields:

Field	Type	Description
movie_id	int	Sent by the client in the POST request body. Acts as the primary key in the database.
title	str	Title of the movie
director	str	Director's name
genre	str	One of: "action", "comedy", "drama", "thriller"
duration	int	Total runtime of the movie in minutes
rating	float	Movie rating (e.g. 4.5)

Database Setup

Create a SQLite database file named movies.db in your project folder. On application startup, create a movies table.

Pydantic Models

Define Pydantic models in a models.py (or schemas.py) file and use them to validate every request body. At minimum:

- MovieCreate — used for POST, requires all fields including movie_id.
- MovieUpdate — used for PUT, requires all updatable fields (movie_id is taken from the path, not the body).

- genre must only accept one of: "action", "comedy", "drama", "thriller" — reject anything else with a 422 response.
- duration must be a positive integer.
- rating must be a float between 0 and 5 (inclusive).

Let FastAPI's automatic request validation do the work — if a client sends an invalid genre, a negative duration, or a missing field, FastAPI should return a 422 error on its own. You do not need to write manual if-checks for this.

Required Endpoints

1. GET /movies

GET	/movies	10 Marks

Return all movies currently stored in the SQLite database.

2. GET /movies/{movie_id}

GET	/movies/{movie_id}	10 Marks

Return a single movie matching the given movie_id.

- If found: return the movie object with status 200.
- If not found: return a meaningful error message with status 404. Do not crash.

3. POST /create_movies

POST	/create_movies	20 Marks

Add a new movie to the collection.

- The client sends all fields in the request body, including movie_id, validated through your MovieCreate Pydantic model.
- Insert the new movie as a row into the SQLite database.
- If a movie with the same movie_id already exists, return a meaningful error with status 400. Do not crash.
- Return the newly created movie object with status 201.

4. PUT /movies/{movie_id}

PUT	/movies/{movie_id}	20 Marks

Update an existing movie's details.

- The client sends the updated fields in the request body, validated through your MovieUpdate Pydantic model.
- Update the corresponding row in the SQLite database.
- If found: apply the update and return the updated movie object with status 200.
- If not found: return a meaningful error message with status 404. Do not crash.

5. DELETE /movies/{movie_id}

DELETE	/movies/{movie_id}	10 Marks

Delete a movie from the collection.

- Delete the matching row from the SQLite database.
- If found: delete it and return a confirmation message with status 200.

- If not found: return a meaningful error message with status 404. Do not crash.

6. GET /movies/sort

GET	/movies/sort	10 Marks

Return all movies sorted by a field, in a given direction.

Query Parameters:

Parameter	Allowed Values	Default
sort_by	duration, rating	rating
order	asc, desc	desc

Example: /movies/sort?sort_by=duration&order=asc

Marking Summary

Task	Method	Endpoint	Marks
1	GET	/movies	10
2	GET	/movies/{movie_id}	10
3	POST	/create_movies	20
4	PUT	/movies/{movie_id}	20
5	DELETE	/movies/{movie_id}	10
6	GET	/movies/sort	10
		Database & Pydantic Validation (correctness across all endpoints)	20
		Total	**100**